FINAL CYBERSECURITY PROJECT

Mapping Authentication and Injection Attacks to OWASP Defensive Controls

Student: Egbon Emmanuel

Project type: Individual Research-and-Technical Investigation

Basis of investigation: Authorized Operation Secure Core Week 4 laboratory evidence, supplemented by independent verification against OWASP guidance.

1. Executive Summary

This project investigates the defensive controls that would have prevented four security weaknesses demonstrated during the authorized Operation Secure Core Week 4 laboratory: token hijacking through OAuth authorization-code interception and replay, an SSO authorization logic flaw, SQL injection, and Redis exploitation. The assigned project title is *Mapping Authentication and Injection Attacks to OWASP Defensive Controls*. The project uses the Week 4 attack evidence as the starting point and independently verifies the defensive mappings against OWASP guidance. The assigned project description explicitly identifies token hijacking, SSO bypass, SQL injection, and Redis exploitation as the four attack cases to be mapped.

The central finding is that the four attacks do not have one universal defence. The strongest mapping is: token hijacking/replay to A07 Identification and Authentication Failures with PKCE, short-lived single-use authorization codes and secure session handling; SSO bypass primarily to A01 Broken Access Control because the demonstrated weakness allowed elevation of privilege through attacker-controlled identity input; SQL injection to A03 Injection through parameterized queries and separation of data from SQL commands; and unauthenticated Redis exposure to A05 Security Misconfiguration through service hardening, authentication, least privilege and network restriction.

The investigation also highlights an important distinction between authentication and authorization. A user can possess a valid authentication token and still receive unauthorized privileges if the server trusts a client-supplied user identifier. OWASP describes elevation of privilege and parameter tampering as examples of broken access control and states that access-control decisions must be enforced in trusted server-side code.

The report therefore evaluates not only whether an AI-generated mapping sounds reasonable, but whether the proposed control actually addresses the observed failure mode. This independent verification is central to the project because the Week 4 exercise explicitly required AI-assisted mapping followed by verification.

2. Project Problem

Modern applications commonly combine authentication, authorization, APIs, databases and supporting infrastructure. A weakness in any one layer can undermine the security of the complete system. The Week 4 lab demonstrated this layered nature of application security: the attack path moved from reconnaissance to credential/code interception, replay and privilege escalation, and later to database and Redis exploitation. The Week 4 Class 2 material specifically describes manual UNION-based SQL injection against a vulnerable /search endpoint and unauthenticated Redis access as part of the authorized exercise.

The problem addressed by this project is therefore the difficulty of selecting the *specific* defensive control that directly breaks each demonstrated attack path. Generic advice such as “improve security” is insufficient. A useful defensive mapping must connect the observed weakness, the attacker capability, the relevant OWASP category/control, and a concrete preventive action.

3. Research Question

Which specific OWASP defensive control would have prevented or materially reduced each of the four Week 4 attacks—token hijacking, SSO bypass, SQL injection, and Redis exploitation—and how can each mapping be independently verified against the observed attack path?

4. Objectives

1. Document the four authorized Week 4 attack cases and the security weakness demonstrated by each.

2. Map each attack to the most directly applicable OWASP Top 10 defensive category/control.

3. Independently verify the AI-assisted mappings against authoritative OWASP guidance.

4. Distinguish authentication failures from authorization failures where the attack path crosses both areas.

5. Evaluate practical preventive measures and recommend controls that can be tested in a development/laboratory environment.

5. Scope

In scope: the four attack cases named in the individual assignment; evidence from the authorized Week 4 Operation Secure Core laboratory; defensive mapping to OWASP Top 10:2021; independent verification of those mappings; risk analysis; and practical recommendations.

Out of scope: testing systems outside the authorized lab, attacking third-party infrastructure, production exploitation, credential theft from real users, use of real personal data, and uncontrolled destructive testing. The Week 4 class materials explicitly limited the exercise to the student's own lab.

6. Methodology

The investigation followed a repeatable evidence-to-control method: (1) identify the attack and its observable security consequence; (2) isolate the root weakness; (3) formulate an initial AI-assisted mapping; (4) independently compare that mapping with OWASP documentation; (5) determine whether the OWASP category directly describes the weakness; (6) identify a concrete preventive control; and (7) record evidence, limitations and residual risk.

The Week 4 material provides direct technical procedures for the authentication cases. For example, the OAuth exercise captures an authorization code, exchanges it at the token endpoint and uses the resulting access token against a protected profile endpoint.
exercise supplies a normal student token and then submits a different user_id to /validate, with the expected vulnerable response identifying an admin role.

For the data-layer cases, the Week 4 Class 2 material identifies a UNION-based SQL injection and unauthenticated Redis access as the practical exercises.

7. Ethical and Safety Controls

All technical testing is restricted to the authorized training environment. Evidence is limited to fictional, synthetic, instructor-provided, or otherwise authorized information. Secrets and credentials are not reproduced in this final report. Testing is stopped if a target is found to be outside the lab boundary or if an action could affect non-lab systems.

8. Investigation Environment and Evidence Sources

The primary evidence base is the student's Week 4 security dossier. Its repository structure contains reconnaissance evidence, OAuth token-hijacking evidence, an AI defensive-mapping journal and a security reflection. The Week 4 README records the attack chain and identifies the vulnerable SSO statement as `const requestedId = req.body.user_id;`.

The Week 4 Class 1 material specifies Nmap, Burp Suite and curl as the principal tools. Its reconnaissance phase targets ports 9000, 3001, 3002, 5432 and 6379; the expected map identifies the two Node.js/Express services, PostgreSQL, Redis and the Medusa HTTP service.

The student's later lab verification also demonstrated a SQL injection effect against the local Medusa /search endpoint. The tested request returned an application record rather than an empty result, showing that the input altered the query outcome in the authorized lab. This result is retained as technical evidence for the SQL injection case; no production or third-party system was involved.

Evidence handling principle: only sanitized observations are reproduced in this report. Authorization tokens, authorization codes, secrets and other credentials are omitted or replaced with placeholders. This follows the final-project submission rule prohibiting private keys, passwords, API keys, access tokens, .env files and other sensitive material.

9. OWASP Defensive Mapping Matrix

Attack	Observed weakness	Primary OWASP mapping	Preventive control
Token hijacking / replay	Authorization code can be intercepted and exchanged before it becomes unusable.	A07 — Identification and Authentication Failures	PKCE; bind code to client/redirect URI; short-lived single-use codes; TLS
SSO bypass	Server trusts client-supplied user_id for identity/privilege decision.	A01 — Broken Access Control	Use verified token claims; enforce server-side authorization and ownership/role checks
SQL injection	Untrusted search input reaches SQL query structure/data without adequate separation.	A03 — Injection	Parameterized queries/safe ORM APIs; positive server-side validation
Redis exploitation	Redis is exposed without authentication and can be read/written by an attacker who can reach it.	A05 — Security Misconfiguration	Require authentication; bind to trusted interfaces; network segmentation; least privilege; hardened configuration

The matrix intentionally distinguishes the Week 4 AI mapping from the independently verified mapping. The class material suggested A07 for the SSO bypass, but OWASP's A01 guidance more directly describes elevation of privilege, parameter tampering and server-side enforcement of access controls.

rather than simply accepting an AI answer.

10. Case Study 1 — Token Hijacking

10.1 Attack path

The Week 4 Class 1 exercise describes an OAuth authorization flow in which an authorization code is visible in a callback request. The class material gives the illustrative intercepted request as a GET to /callback containing a code and state parameter, and instructs the student to save the captured code before it expires.

The next phase exchanges the intercepted authorization code for an access token. The resulting token is then presented to a protected profile endpoint. The expected lab result identifies the authenticated profile as admin.

10.2 Security weakness

The security weakness is insufficient protection and binding of an OAuth authorization code. If an attacker can capture a valid code and replay it successfully, possession of the temporary code becomes sufficient to obtain a bearer access token.

10.3 OWASP verification

OWASP A07 includes authentication and session-management weaknesses and maps CWE-294, Authentication Bypass by Capture-replay, to the category.
material independently names PKCE, code binding and short expiry as the common mapping for code replay under A07.

TLS is also important because it protects confidentiality and integrity of traffic. OWASP's Transport Layer Security guidance states that TLS provides confidentiality and integrity for application traffic.

code binding address the replayability of the authorization code itself.

10.4 Finding

Finding: The most direct OWASP Top 10 mapping is A07, with PKCE/code binding and short-lived single-use authorization codes as the principal controls. TLS reduces the opportunity for interception but does not by itself replace code binding.

11. Case Study 2 — SSO Authorization Bypass

11.1 Attack path

The lab creates a normal student authentication token and then sends that token to /validate while supplying a different user identifier in the request body. The expected vulnerable response is valid=true with an admin identity.

The teaching material explicitly identifies the root cause: `const requestedId = req.body.user_id;` trusts client input when the server should instead rely on verified token claims.

11.2 Independent OWASP verification

The initial AI journal mapped this flaw to A07. However, independent review of OWASP A01 shows a more precise classification: A01 covers bypassing access controls by modifying request parameters, permitting access to another user's account, and elevation of privilege. OWASP further states that access control is effective only when enforced in trusted server-side code.

11.3 Finding

Finding: A01 Broken Access Control is the strongest primary mapping for the SSO bypass demonstrated in Week 4. A07 remains relevant to the authentication/session context, but the immediate security failure is an authorization decision based on attacker-controlled identity data.

11.4 Recommended control

The endpoint should derive the authenticated subject from the verified token/session rather than accepting a user_id that determines the identity. Authorization must then be evaluated server-side for every protected operation. Functional authorization tests should include attempts to change object identifiers and roles.

12. Case Study 3 — SQL Injection

12.1 Attack path

The Week 4 Class 2 material identifies a UNION-based SQL injection in the /search endpoint and states that the exercise is performed against the student's own lab.

During the subsequent authorized local verification, a baseline search returned an empty JSON array. A modified query containing a SQL condition returned the lab's sample product record. The important observation is the change in application behavior caused by crafted input; this is consistent with the seeded SQL injection exercise described in the class material.

12.2 OWASP verification

OWASP A03 Injection explicitly includes SQL injection and explains that applications become vulnerable when user-supplied data is not adequately separated from SQL commands or when dynamic, non-parameterized queries are constructed with hostile input. OWASP recommends safe APIs and parameterized interfaces/ORMs, supported by positive server-side input validation.

12.3 Finding

Finding: SQL injection maps directly to A03 Injection. The primary preventive control is parameterized query construction or a safe ORM/database API that keeps data separate from query structure.

12.4 Validation approach

A secure implementation should be tested with both normal search terms and adversarial strings. The expected secure result is that the input remains data and cannot change SQL logic. Regression tests should cover quotes, boolean expressions, UNION-like input and other metacharacters appropriate to the application's query interface.

13. Case Study 4 — Redis Exploitation

13.1 Attack path

The Week 4 Class 2 material describes connecting to Redis without a password, reading and writing keys, and explaining the SLAVEOF escalation path. It emphasizes that the exercise is limited to the student's own lab.

13.2 Security weakness

The root weakness is an insecurely exposed supporting service: Redis accepts unauthenticated access from an attacker who can reach the service. Read/write access to a data store creates a path to data disclosure, modification and potentially further compromise depending on the service's configuration and surrounding privileges.

13.3 OWASP verification

OWASP A05 Security Misconfiguration covers missing hardening, unnecessary exposed services, insecure settings and the need for repeatable secure configuration across application components.

level because the primary weakness is unsafe service configuration/exposure.

13.4 Finding

Finding: Redis exploitation is primarily A05 Security Misconfiguration when the enabling condition is unauthenticated and unnecessarily reachable Redis. The strongest mitigation is defense in depth: authentication/ACLs, network restriction, least privilege, secure binding and removal of unnecessary exposure.

13.5 Evidence limitation

The available Week 4 project repository documents the Redis exploitation as part of the course exercise, but the repository contents reviewed for this report did not contain a dedicated Redis screenshot/log. Therefore, this report does not invent a Redis command output or claim a specific result beyond what the course material documents.

14. Findings and Analysis

The four cases demonstrate four different failure modes across the application stack.

Layer	Failure	Security property affected	Primary OWASP
Identity/session	Authorization code replay enables token acquisition	Authentication/session integrity	A07
Authorization	Client-controlled user_id influences privileged identity	Authorization / privilege separation	A01
Database	SQL syntax can be influenced by untrusted input	Confidentiality and integrity of data	A03
Infrastructure	Redis accepts unauthenticated access	Confidentiality, integrity, availability	A05

A major analytical result is that “authentication” and “authorization” must not be treated as synonyms. The SSO case begins with a valid student token, so authentication succeeds. The vulnerability appears afterward when the application accepts a different identity from the request body. OWASP A01 directly identifies elevation of privilege and parameter tampering as broken access-control scenarios.

A second result is that defensive controls operate at different layers. TLS protects data in transit; PKCE and code binding reduce the usefulness of a captured OAuth code; server-side authorization protects identity-to-resource decisions; parameterized SQL protects the database interpreter; and service hardening/segmentation protects infrastructure. A mature security design therefore combines controls rather than relying on a single mechanism.

15. Risk Considerations

The risk assessment considers impact and likelihood within the authorized laboratory context. It is not a production risk rating.

Attack	Potential impact	Likelihood if weakness remains	Priority
Token hijacking	Session/account impersonation and unauthorized access	High where codes can be intercepted/replayed	High
SSO bypass	Privilege escalation and unauthorized account access	High when identity is client-controlled	Critical
SQL injection	Database disclosure, modification or destruction	High where query construction is vulnerable	Critical
Redis exploitation	Data manipulation and possible downstream compromise	High if unauthenticated and reachable	High

The highest-priority risks are the SSO authorization flaw and SQL injection because both can directly cross trust boundaries: one can turn a lower-privileged identity into a higher-privileged one, while the other can turn an application input into database instructions. Token replay and Redis exposure also deserve high priority because compromise can extend beyond the initially affected component.

16. Recommendations

1. **OAuth:** implement PKCE for applicable authorization-code flows, bind authorization codes to the legitimate client/redirect context, enforce short expiration, and reject reuse.

2. **Transport:** use HTTPS/TLS for authentication and authorization traffic. OWASP recommends TLS for confidentiality and integrity of application communications.

3. **SSO/authorization:** derive the subject from verified authentication claims and perform server-side authorization for every protected operation. Never let a request-body user_id override the authenticated identity.

4. **SQL:** replace dynamic SQL concatenation with parameterized queries or safe ORM/database APIs. OWASP's A03 guidance identifies keeping data separate from commands as the core prevention principle.

5. **Redis:** require authentication/ACLs, restrict network reachability, bind Redis to trusted interfaces, remove unnecessary exposure, and apply least privilege.

6. **Testing:** add regression tests for authorization parameter tampering, OAuth code reuse, malicious SQL input and unauthenticated access to infrastructure services.

7. **Monitoring:** log failed authorization decisions, suspicious token use, repeated authentication failures and unexpected access to infrastructure services. OWASP A01 also recommends logging access-control failures and alerting administrators when appropriate.

8. **Evidence and repository hygiene:** keep only sanitized screenshots, logs and configuration in the public repository. Never commit tokens, secrets, .env files, private keys or real identity documents.

17. Limitations

The investigation has several limitations. First, it is based on an authorized training environment rather than a production system. Second, the Week 4 repository available for review contains dedicated evidence for reconnaissance and OAuth token hijacking, but no dedicated Redis evidence file was present in the repository listing reviewed during preparation. Third, the final-project evidence set should ideally contain a dedicated screenshot or log for each of the four attack cases. This report therefore distinguishes documented course evidence from directly reproduced observations and does not fabricate missing evidence.

The SQL injection observation was reproduced in the authorized local environment during project preparation, but a full source-code review of the exact vulnerable /search implementation was not available in the Week 4 dossier itself. Consequently, the report relies on the Week 4 lab description for the vulnerability characterization and on OWASP's A03 guidance for the independent control verification.

The OWASP mapping is also a classification exercise rather than a claim that one category is the only possible classification. For example, the Week 4 AI journal associated the SSO bypass with A07, while independent review supports A01 as the stronger primary classification because the demonstrated outcome is privilege escalation through a client-controlled identifier.

18. Conclusion

This project demonstrates that defensive security mapping is most useful when it is tied to an actual attack path. The four Week 4 cases reveal different control requirements: OAuth code replay requires protection of the authorization flow and replay resistance; the SSO flaw requires trusted server-side authorization decisions; SQL injection requires separation of data from SQL commands; and Redis exploitation requires secure service configuration and restricted exposure.

Independent OWASP verification strengthened the original AI-assisted analysis. In particular, the SSO case shows why AI output should be treated as a starting hypothesis rather than final authority: A07 is relevant to authentication/session concerns, but A01 more directly captures the demonstrated broken authorization and privilege escalation. OWASP A03 directly covers SQL injection, while A05 covers insecure service configuration and hardening.

The overall conclusion is that secure authentication alone does not guarantee secure authorization, and secure application code alone does not guarantee secure infrastructure. Effective defence requires controls at the identity, authorization, application, database and infrastructure layers, together with repeatable security testing and evidence-based verification.

19. References

1. OWASP Foundation. *OWASP Top 10:2021*. Official OWASP documentation. https://owasp.org/Top10/2021/

2. OWASP Foundation. *A01:2021 — Broken Access Control*. Official OWASP documentation. https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/

3. OWASP Foundation. *A03:2021 — Injection*. Official OWASP documentation. https://owasp.org/Top10/2021/A03_2021-Injection/

4. OWASP Foundation. *A05:2021 — Security Misconfiguration*. Official OWASP documentation. https://owasp.org/Top10/en/A05_2021-Security_Misconfiguration/

5. OWASP Foundation. *A07:2021 — Identification and Authentication Failures*. Official OWASP documentation. https://owasp.org/Top10/2021/A07_2021-Identification_and_Authentication_Failures/

6. OWASP Foundation. *Transport Layer Security Cheat Sheet*. OWASP Cheat Sheet Series.

7. Operation Secure Core — Week 4 Class 1. Instructor-provided laboratory material: Kali Recon, Token Hijacking & SSO Bypass. Used as authorized technical evidence and procedural context.

8. Operation Secure Core — Week 4 Class 2. Instructor-provided laboratory material: Manual SQL Injection & Redis Exploitation. Used as authorized technical evidence and procedural context.

9. Student Week 4 Security Dossier: *operation-secure-core-week4*. Repository evidence supplied for this investigation.

20. Evidence Register and Final Submission Checklist

The final report should be accompanied by the evidence and project files in the final public GitHub repository. The table below records what should be present before submission.

Evidence	Purpose	Status at report preparation
Reconnaissance screenshot	Shows the authorized attack surface/service map	Available in Week 4 dossier
OAuth interception/replay evidence	Shows authorization-code capture and resulting protected identity	Available in Week 4 dossier
SSO bypass evidence	Shows normal token producing unauthorized admin identity	Documented by class evidence; dedicated screenshot should be added to final repo if available
SQL injection evidence	Shows changed application result from crafted input	Local verification observed during preparation
Redis evidence	Shows unauthenticated Redis exposure/read-write capability	Course material documents exercise; dedicated lab output should be added if available
AI verification journal	Records initial AI mapping and independent verification	Drafted in final project; preserve permitted AI disclosure
README	Explains project purpose, structure, reproduction and evidence	Included in project package

Submission requirements to verify before LMS upload: one PDF under 10 MB; approximately 10–20 pages; public GitHub repository; correct repository link inside the report; repository opens while

acknowledged; and permitted AI assistance disclosed where required.

GitHub repository: https://github.com/festusemirated666-byte/Final-project

AI assistance disclosure: This project used AI assistance for initial defensive mapping and drafting support. The OWASP mappings were independently checked against authoritative OWASP documentation, and the final report distinguishes the AI starting point from the verified analysis.